Heart Risk: Reproducible Classification

AIMLCZG523 | Assignment 01 | End-to-end MLOps

Author: Ayush Kumar, 2025AE05258

This project treats a machine-learning result as a traceable software release. Its objective is to classify recorded heart-disease presence from the UCI Cleveland clinical measurements and serve the selected classifier through a validated, monitored API. Acquisition, feature processing, experiment comparison, packaging, automated checks, and deployment are explicit stages rather than ad hoc notebook actions.

The solution uses two established tabular classifiers: Logistic Regression and Random Forest. They share a deterministic holdout split, identical cross-validation folds, and fold-local preprocessing. Selection is based only on mean validation ROC-AUC. The untouched holdout is used for the final comparison after the selection decision is fixed. The small difference between the models' CV means does not demonstrate statistical superiority.

The deployment target is local Docker Desktop Kubernetes, an option explicitly permitted by the assignment. The architecture supports multiple stateless API replicas while keeping experiment tracking outside the prediction path. GitHub Actions is responsible for validation, training, container verification, and continuous delivery of a tested image. A local release command performs the deployment because hosted runners cannot reach a laptop's localhost.

This is an educational classifier, not a validated clinical device. The target describes disease status in the source dataset; it does not establish prospective cardiovascular risk. A returned probability is a model output, not an individualized medical guarantee. No deployment, CI run, screenshot, or recording is claimed unless its actual evidence exists.

Selected model	CV ROC-AUC	Holdout recall
logistic_regression	0.911	0.929

Architecture and Reproduction

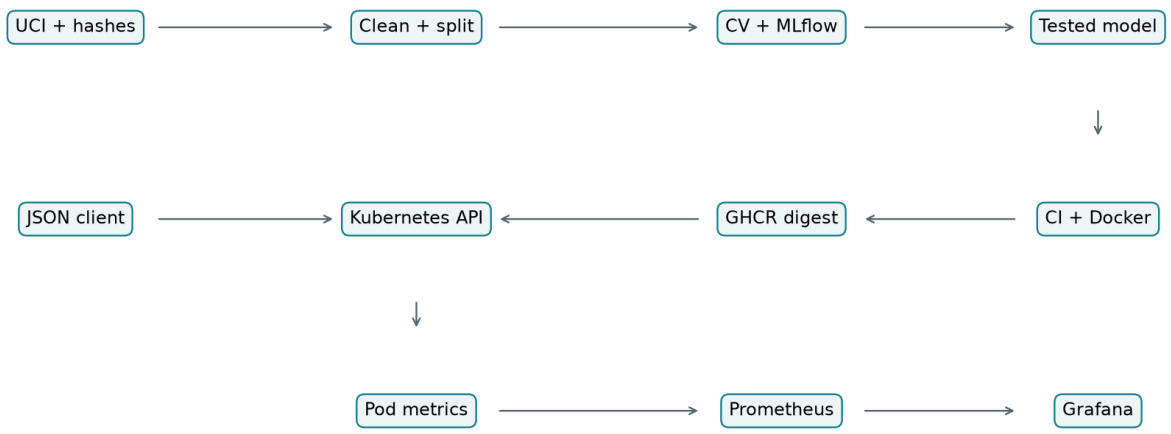
Data lineage, artifact boundaries, and environment isolation

The raw UCI snapshot is checked against a SHA-256 digest. A canonical cleaned CSV and split manifest then define the input to EDA and training. Model training exports one preprocessing-plus-classifier Pipeline, metadata, evaluation outputs, and MLflow runs. The serving application loads the trusted artifact once at startup and never trains, fetches data, or downloads a replacement model during inference.

Python 3.11 is used consistently across the Windows development environment, Ubuntu CI, and Linux container. The full dependency lock contains exact versions and distribution hashes. A smaller serving lock is constrained by that same environment, preventing accidental sklearn, NumPy, pandas, or joblib skew. Installation uses a fresh virtual environment, the requirements lock, and an explicit package install.

The reproduction order is environment installation, lint and tests, data preparation, EDA, training, offline inference, and API smoke verification. The README contains executable Windows commands and deployment instructions. The supplied PowerShell wrapper checks native process exit codes so a failed command cannot be hidden by a later successful step.

Generated MLflow storage is local and is not a shared production database. Uploaded experiment artifacts preserve the run evidence; metrics JSON, CSV tables, figures, and model bundles are portable. Absolute filesystem artifact paths in a copied SQLite database are not assumed to relocate automatically.



Original architecture: tested artifacts flow toward serving; telemetry flows separately.

Data Provenance and Cleaning

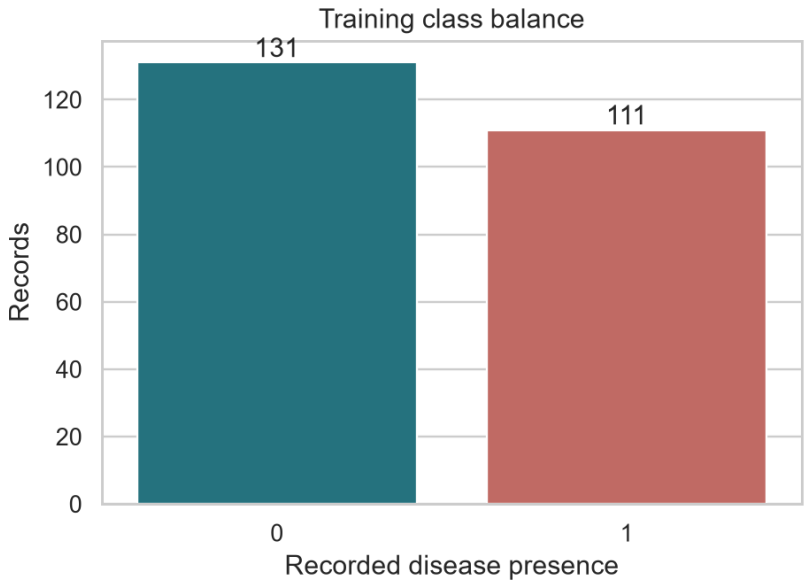
Official Cleveland subset; no third-party remapping

The source is the UCI Machine Learning Repository Heart Disease archive, specifically processed.cleveland.data. It contains 303 records and 13 predictors plus the num target. Although the assignment says 14+ features, the canonical processed subset has 14 columns including the target. One-hot encoding expands the model matrix; no synthetic patient measurements are invented to satisfy the wording.

The source target ranges from zero to four. Zero is mapped to absence and values one through four to presence. Original category codes are preserved: chest-pain type is 1-4, slope is 1-3, thal uses 3/6/7, and ca uses 0-3. Mixing these values with zero-indexed codes from republished CSVs would change their meaning, so both ingestion and API validation enforce the original contract.

The parser rejects malformed rows, invalid labels, unsupported categories, and non-finite values. Question marks become missing values. The official snapshot contains four missing ca entries and two missing thal entries. No duplicate records were found. Exact duplicates would be removed deterministically; identical predictors with conflicting targets raise an error for inspection.

The cleaned CSV deliberately retains missing feature values. Imputing the entire dataset before splitting would leak holdout or validation statistics. A stratified 80/20 split with seed 42 yields 242 training records and 61 test records. Checksums, row indices, schema version, class counts, and cleaning decisions are stored in data manifests. Dataset redistribution is attributed under CC BY 4.0.



Class balance is shown for the training partition; full-source counts are 164 absent and 139 present.

EDA and Feature Processing

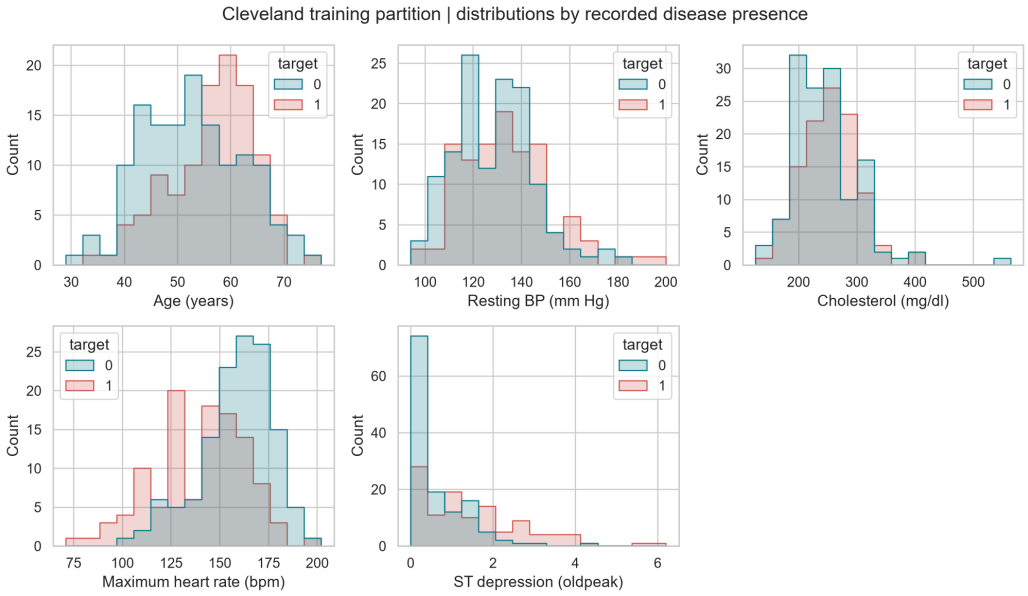
Training-only interpretation and fold-local transformations

EDA uses only the training partition for modelling decisions. Outputs include distributions with disease-presence overlays, a numeric Pearson-correlation heatmap, class balance, missingness counts, and chest-pain category counts. Numeric correlations are not presented as correlations between arbitrarily numbered nominal categories. Every plot identifies its population and original units where applicable.

Age, resting blood pressure, serum cholesterol, maximum achieved heart rate, and oldpeak form the numeric block. A median imputer precedes StandardScaler. The remaining categorical or discrete fields use most-frequent imputation followed by one-hot encoding. This avoids telling Logistic Regression that thal code 6 has twice the magnitude of code 3. ca is treated as a small categorical count so its effect need not be linear.

A ColumnTransformer and sklearn Pipeline bind preprocessing to the classifier. GridSearchCV clones and fits that complete pipeline inside each training fold. Unseen categories are handled by the encoder, while the API rejects category values outside the documented data contract. Unit tests check learned numeric statistics against the training slice and confirm that test-time transformation does not mutate them.

The first experiment retains all 13 source predictors rather than performing unstable feature selection on this small sample. No target-derived features, global imputations, oversampling before cross-validation, or test-driven threshold adjustments are used. Histograms and categorical comparisons are descriptive associations, not evidence of causal clinical effects.



Distributions generated from the fixed training set; missingness and correlation figures accompany the repository.

Model Selection and Evaluation

Shared folds, bounded searches, and a locked decision threshold

Logistic Regression explores C in {0.1, 1, 10} and class_weight in {None, balanced}: six candidates. Random Forest explores three depths, two minimum-leaf sizes, and two class-weight settings with 200 trees: twelve candidates. Both use the same five shuffled stratified folds and seed. This bounded search makes the comparison auditable and inexpensive enough to rerun in CI.

The selection criterion is mean training-validation ROC-AUC, with an exact numerical tie favoring Logistic Regression. Accuracy, precision, recall, F1, and ROC-AUC are retained for the best candidate from each family, with fold standard deviations. A prior-only DummyClassifier gives a ROC-AUC reference of 0.5. Cross-validation variability is reported rather than implying that the highest point estimate is definitive.

The classifier family and hyperparameters are locked before holdout evaluation. A threshold of 0.5 is fixed in configuration. Both family winners are evaluated for the final comparison, but their holdout results cannot change the selected champion. Precision and recall are included because false positives and false negatives carry different costs in a medical-screening context; this assignment does not establish those costs clinically.

The test set has only 61 observations. High holdout scores therefore should not be treated as an externally validated performance guarantee. Repeating the deterministic run is a reproducibility check, not permission to iterate parameters against test outcomes. Additional multicentre, prospective, calibration, and subgroup validation would be necessary before any clinical application.

Metric	Logistic Regression	Random Forest
Holdout accuracy	0.885	0.869
Holdout precision	0.839	0.812
Holdout recall	0.929	0.929
Holdout f1	0.881	0.867
Holdout roc_auc	0.965	0.953
CV ROC-AUC mean (SD)	0.911 (0.030)	0.909 (0.035)

Experiment Tracking and Packaging

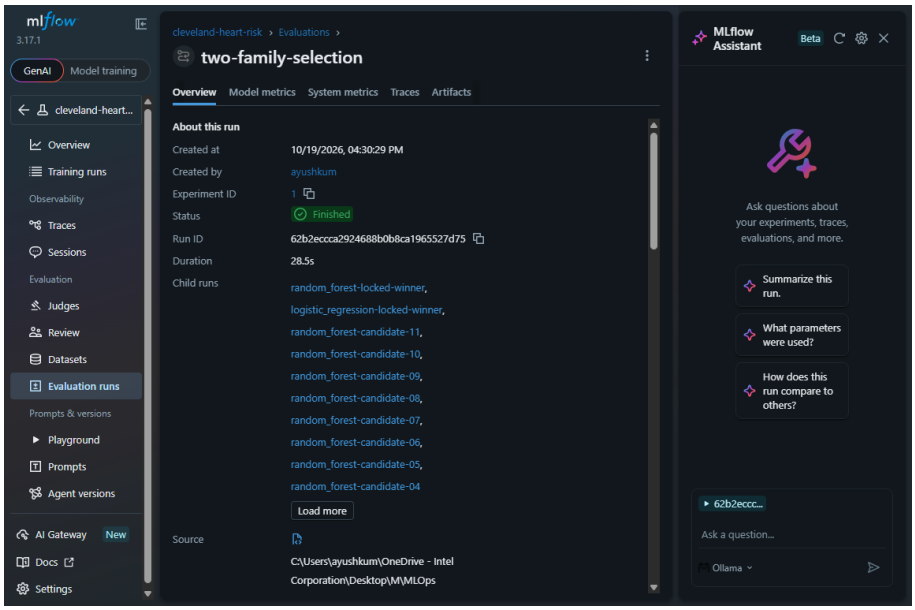
Inspectable MLflow runs and trusted model persistence

MLflow uses an explicitly configured local SQLite tracking URI and filesystem artifact root. A parent run records shared preprocessing choices, seed, fold count, data and split manifests, environment versions, and final results. Each candidate has a child run with parameters, fold scores, metric mean/standard deviation, and a CV ROC-AUC plot. Each family winner also has a logged reusable model and evaluation plot.

The serving bundle contains pipeline.joblib and metadata.json. Metadata records the schema, feature order, classes, threshold, model checksum/version, software versions, data/split/config hashes, MLflow run ID, and source identity. Inference verifies the checksum and core dependency versions before deserialization, then checks the fitted model's feature names and class order. A checksum checks integrity; it is not a substitute for trusting the artifact producer.

The first local tracking attempt exposed a default MLflow/skops compatibility issue involving numpy.dtype. The run failed visibly. The corrected implementation explicitly logs the locally trained model using cloudpickle and retains the failed run as diagnostic evidence. No model metric or tuning setting was changed to resolve packaging. Pickle, joblib, and cloudpickle artifacts must never be loaded from untrusted sources.

Serialization is tested against the original fitted pipeline, and the live API is checked against offline inference. The selected training artifact is embedded into the container instead of retrained at startup. This creates a direct link between the evaluated model and the model served by a release.



Real local MLflow evidence; a missing capture remains explicitly pending.

Automated Tests and CI/CD

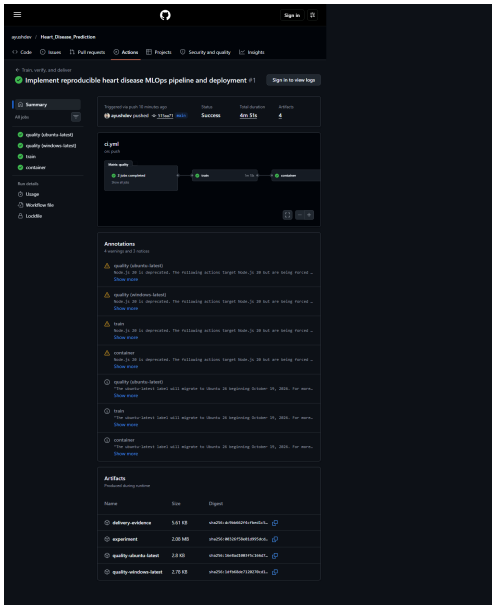
Failure propagation and delivery of the same tested image

Pytest covers parsing, label mapping, invalid categories, missing values, checksum failures, disjoint reproducible splits, model determinism, serialization, train-only statistics, API validation, probability semantics, privacy-conscious logs, and delivery configuration. Small deterministic fixtures and mocked acquisition failures keep unit tests independent of the network and a real cluster. Full training uses the pinned real dataset as a separate integration gate.

GitHub Actions validates Python on Windows and Ubuntu, checks the hash-locked environment, runs Ruff lint/format checks and pytest, and uploads test reports. A dependent Linux job prepares the data, generates EDA, trains both classifiers, logs MLflow, and uploads model/evaluation artifacts. The container job downloads that bundle, builds the API image, and smoke-tests it without host model mounts.

Only trusted pushes to main may publish the already-tested image to GHCR. Authentication uses the workflow's scoped GITHUB_TOKEN, not a committed password. The release manifest records the registry digest, model hash, source revision, and CI URL. Pull requests cannot publish. Action dependencies are pinned to commit SHAs, and required stages do not use continue-on-error.

Artifact and log collection uses always-run steps without weakening test gates. Cleanup may tolerate an absent test container, but lint, tests, training, build, smoke verification, and publishing do not mask failures. Delivery to GHCR is automated; local Kubernetes deployment is an explicit user-invoked release step because the cluster is not routable from hosted runners.



Successful hosted run 39396167399: cross-platform checks, training, isolated serving, and image publication.

API and Container Isolation

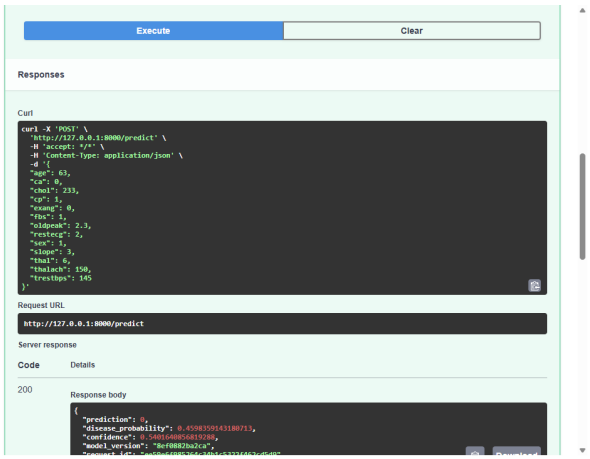
Validated requests, deterministic inference, and runtime boundaries

POST /predict accepts one JSON record with exactly the 13 original feature names. Numeric values must be finite and in their basic positive/nonnegative domains. ca and thal may be explicitly null; missing keys and invalid original-UCI categories are rejected with HTTP 422. Extra fields and implicit string/boolean coercions are rejected so callers cannot silently change the input contract.

Responses include a binary prediction, disease_probability, confidence, model_version, and request_id. disease_probability is P(class 1); confidence is the probability assigned to the returned class. Prediction uses a fixed 0.5 threshold. These values are not claims of calibrated medical certainty. The local smoke check compares predictions with the trusted offline artifact and exercises an invalid category request.

The Dockerfile uses a digest-pinned official Python 3.11 base, a smaller hash-locked serving environment, and a non-root process. The image includes the pre-trained artifact and necessary source but excludes raw data, tracking storage, virtual environments, credentials, and screenshots. It does not download data or train during build or startup. Health and readiness endpoints support both container checks and orchestration.

The Linux image was built and exercised without mounting the host model directory. Local container predictions matched offline inference, invalid original-UCI codes returned HTTP 422, and metrics were exposed. Hosted CI independently built and verified its image before publishing that same image by digest. The earlier local model is 8ef0882ba2ca; the deployed CI model is 24f056bb0011. Their evaluation metrics and sample probabilities agree.



Actual local API response; screenshots/docker.png and container.json record the separate no-mount Docker check.

Kubernetes and Operational Monitoring

Stateless replicas, explicit exposure, and per-pod telemetry

The API Deployment runs two stateless replicas in a dedicated heart-risk namespace with resource requests/limits, startup/readiness/liveness probes, a non-root security context, and rolling updates. A LoadBalancer Service exposes localhost:8080 on Docker Desktop. The release script explicitly targets docker-desktop, renders manifests through a YAML parser, waits for rollout and service ingress, then checks the model version through the real service endpoint.

Prometheus uses namespace-scoped pod discovery and a Role limited to reading pods. Each API pod is scraped separately, avoiding counter discontinuities from scraping a load-balanced service. The provisioned Grafana dashboard shows request rate, p95 latency, errors by status, prediction counts, and pod scrape availability. Its datasource uses cluster DNS, not a container's localhost.

Request logs contain timestamps, generated request IDs, templated routes, status, elapsed time, and model version. Patient feature bodies and secrets are not logged; unmatched paths use a bounded label. Grafana's password is supplied through a Kubernetes Secret. Monitoring interfaces are accessed through loopback port-forwards rather than the public-facing API Service.

The controlled traffic script sends valid and invalid sample requests, then requires at least two healthy API scrape targets and a populated request-rate series. This verifies operational monitoring, not prediction accuracy or concept drift, which would require delayed ground-truth labels. Monitoring storage is intentionally ephemeral with short retention for this local assignment; a production environment would require durable storage, authentication, TLS, and retention policies.



Controlled traffic: 120 requests, two healthy API pod targets, and verified Kubernetes LoadBalancer inference.

Evidence, Limitations, and Submission

Measured outcomes and an explicit completion boundary

The primary result is a reproducible, traceable classifier-to-API workflow. Logistic Regression and Random Forest both exceed the prior-only validation baseline in this experiment. The selected model's holdout metrics are observations on the fixed test set, not guarantees for new populations. The dataset is small, historical, and clinical rather than a representative prospective screening cohort; categorical definitions and measurement availability constrain external use.

The evidence manifest records genuine screenshot paths, execution-check outputs, repository URL, and video location. The PDF builder marks incomplete reports as DRAFT and refuses final mode when mandatory items are missing. This prevents a deployment manifest, a video script, or a placeholder image from being presented as evidence of execution. Automated page-count validation checks the requested ten-page length.

The final submission includes source, the original and cleaned attributed dataset, dependency locks, EDA/training/inference scripts, tests, GitHub Actions YAML, Dockerfile, Kubernetes and monitoring manifests, screenshots, the final PDF, local access instructions, and a short narrated recording. The student should explain preprocessing leakage prevention, model selection, image identity, failure gates, and metric semantics in their own words and follow the course's rules on assistance disclosure.

References: Janosi, A., Steinbrunn, W., Pfisterer, M., and Detrano, R. (1989), Heart Disease [Dataset], UCI Machine Learning Repository, https://doi.org/10.24432/C52P4X, CC BY 4.0. Implementation references: scikit-learn user guide (pipelines and cross-validation); MLflow tracking documentation; FastAPI documentation; Docker Desktop Kubernetes documentation; GitHub Actions package publishing documentation; Prometheus and Grafana documentation. These sources inform library use, not copied student project implementations.

Repository: https://github.com/ayushdev/Heart_Disease_Prediction

Video: PENDING

Local API: http://localhost:8000/docs; deployed access: http://localhost:8080/docs after successful Kubernetes verification.

Outstanding: actual demo recording or accessible video URL